UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

FLUTTER



JULIO 2026-DICIEMBRE 2026

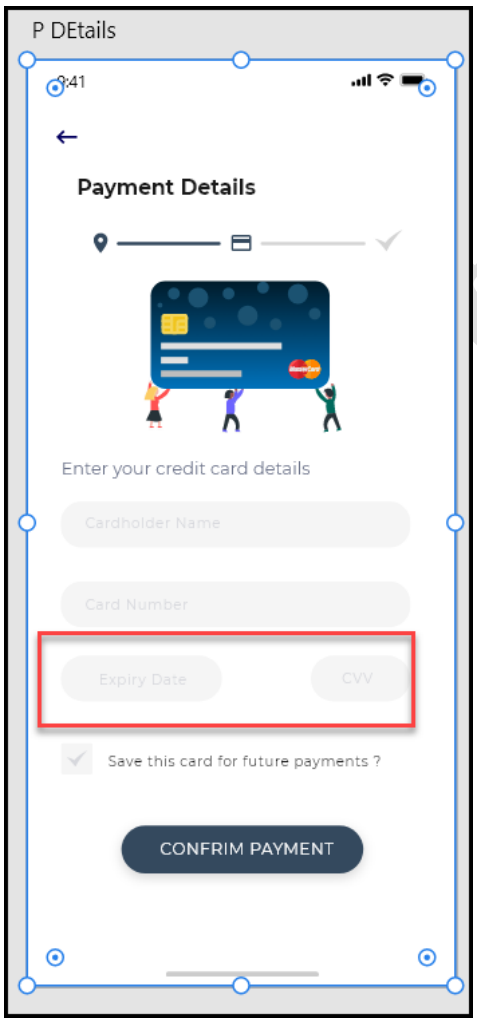
16/10/2026

Ejercicio # 1

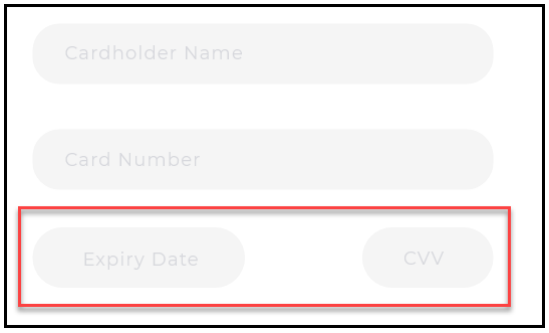
En esta práctica vamos a continuar con el diseño de la segunda pantalla que tiene relación con los detalles de pago para la aplicación en Flutter.

Detalles de pago #4

1. Abra el **mockup** para continuar con el diseño de la segunda pantalla, de acuerdo con el **mockup**, ahora se necesita crear la **UI** para los **siguientes campos** de la pantalla para el ingreso de datos que se muestran en las siguientes figuras.



T-Clay Aldás



2. Como se muestra en la figura anterior los campos **Cardholder Name** (nombre del propietario de la tarjeta) y **Card Number** (número de tarjeta) ocupan **todo el ancho de la pantalla**, sin embargo, los campos que se requiere crear a continuación son **Expiry Date** y **CVV**, los cuales ocupan un **porcentaje** de todo el ancho de la pantalla y no la pantalla completa.

3. Por lo tanto, abra el archivo **payment_details_screen.dart** lo primero que vamos a hacer es **agregar** un **espacio en blanco** al final del **ListView**, se pueden agregar espacios en blanco con varios tipos de componentes, como con un **Padding**, pero nosotros estamos utilizando para esto **SizeBox**. Esto se muestra a continuación.

payment_details_screen.dart M

```
116                   return null;
117                 },
118                 onSaved: (value) {
119                   cardNumber = value!;
120                 },
121                 inputFormatters: <TextInputFormatter>[
122                   MaskTextInputFormatter(mask: '####-####-####-####'),
123                 ], // <TextInputFormatter>[]
124                 keyboardType: TextInputType.number,
125                 focusNode: cardNumberFocus,
126                 textInputAction: TextInputAction.next,
127               ), // TextFormField
128
129               const SizedBox(height: 10),
130
131             ],      You, 7 days ago • Terminado domingo 22 de octubre
132           ), // ListView
133         )), // Padding // Form
134       ); // Scaffold
135     }
136
137     @override
138     void initState() {
139       super.initState();
140
141       cardNumberFocus = FocusNode();
```

4. De acuerdo con el **mockup**, a continuación del **SizeBox** anterior debemos agregar un **widget** de tipo **Row** dentro del cual se ubicará los textos para el **ingreso de datos** de datos para el **Expiry Date** y **CVV**. Cree por lo tanto un widget/clase **Row**.

payment_details_screen.dart M

```
                ], // <TextInputFormatter>[]
                keyboardType: TextInputType.number,
                focusNode: cardNumberFocus,
                textInputAction: TextInputAction.next,
              ), // TextFormField
              const SizedBox(height: 10),

              const Row(children: [
              ]), // Row

            ],      You, 7 days ago • Terminado domingo 22 de octubre
          ), // ListView
        )), // Padding // Form
    ); // Scaffold
  }

  @override
  void initState() {
    super.initState();
```

5. Dentro del **Row** agregue el primer texto (**TextFormField**) con la propiedad **decoration** que utilice el **control personalizado** creado en la práctica anterior para reutilizar el código. (Note la eliminación de **const** del **Row** del código).

payment_details_screen.dart M

```
              const SizedBox(height: 10),

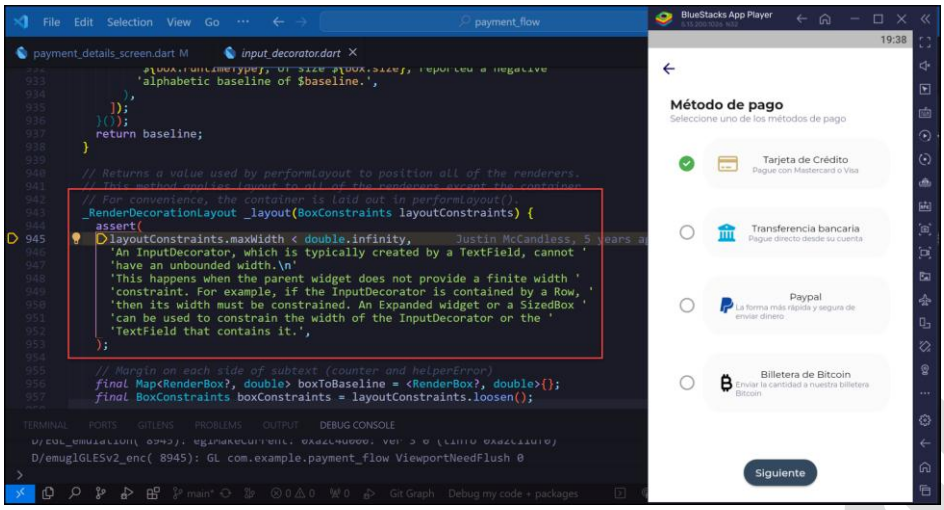
              Row(children: [
                TextFormField(decoration: CustomInputDecoration('Fecha de expiración') ),
              ]), // Row

            ],      You, 7 days ago • Terminado domingo 22 de octubre
          ), // ListView
        )), // Padding // Form
    ); // Scaffold
  }

  @override
  void initState() {
    super.initState();
```

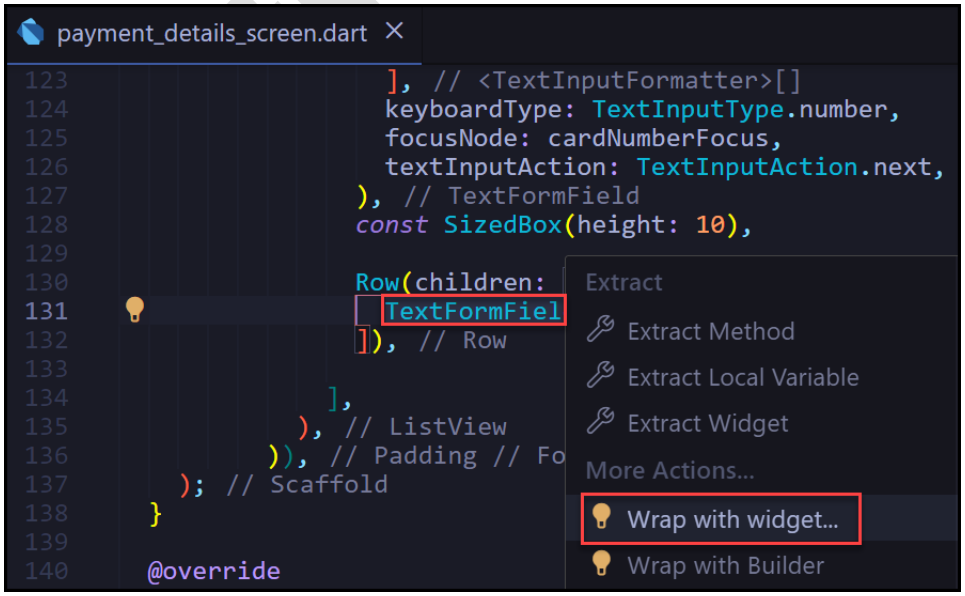
Nota	• Tome en cuenta el uso de la **palabra reservada const** en cada parte del código. En el punto 4 se **agregó const** al **Row** porque es necesario, sin embargo, en el punto 5 es **eliminado** porque el **TextFormField** no puede ser **constante** puesto que cambia el texto que va dentro de este control.

6. **Ejecute** y **pruebe** el proyecto. Esto genera un **error**.



7. Como se puede visualizar en la figura anterior, el **TextFormField** genera un **error**, puesto que este no **tiene un ancho específico** y por lo tanto intenta tomar **todo el espacio disponible** y como el **TextFormField** está dentro del **Row** entonces el **Row** se **acopla a este ancho**, es decir el **Row** busca tener un **ancho infinito** y por lo tanto se **genera un problema**.

8. Para corregir el error **coloque el cursor** dentro del widget **TextFormField** y luego presione la teclas **Control + el punto** (Control + .), de la lista que se muestra **seleccione** la opción **Wrap with widget**.



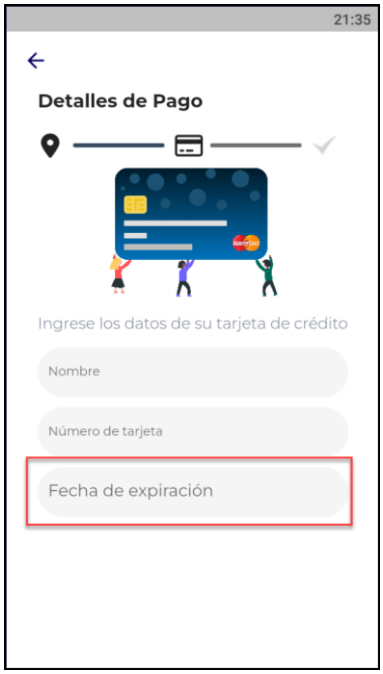
9. Esto genera el **siguiente código**.



10. **Cambie** la palabra **widget** por la palabra **Expanded** para envolver el **TextFormField** con un widget de tipo **Expanded**, esto hace que el **TextFormField** tome **todo el espacio que sobre** del ancho de la pantalla puesto que no hay otro control en junto a este.



11. **Ejecute** y pruebe el proyecto.



Nota	• Si el programa continúa **generando un error**, **detenga totalmente la ejecución del programa** y **ejecute nuevamente**, esto debería corregir el error.

12. A continuación (**No dentro**) del **Expanded** anterior **agregue** un segundo **Expanded** y dentro de este último **Expanded** agregue un **SizeBox** para poner un espacio en lo ancho (width), el código se muestra a continuación.

payment_details_screen.dart M

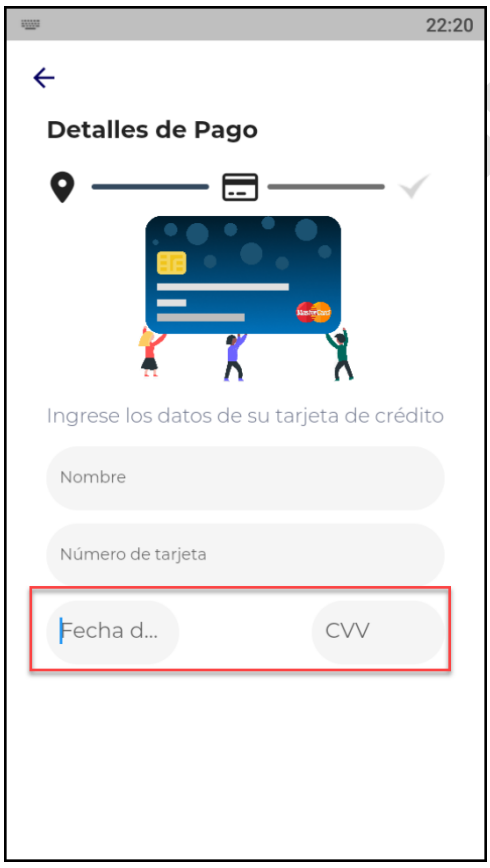
```
122                 MaskTextInputFormatter(mask: '####-####-####-####'),
123               ], // <TextInputFormatter>[]
124               keyboardType: TextInputType.number,
125               focusNode: cardNumberFocus,
126               textInputAction: TextInputAction.next,
127             ), // TextFormField
128             const SizedBox(height: 10),
129             Row(children: [
130               Expanded(
131                   child: TextFormField(
132                       decoration:
133                           CustomInputDecoration('Fecha de expiración'))), // TextFormField
134
135               const Expanded(child: SizedBox(width: 20)),
136
137             ]), // Row
138           ],
139         ), // ListView
140       )), // Padding // Form
141     ); // Scaffold
142   }
```

13. A continuación, agregue un tercer **Expanded** y dentro de este otro **TextFormField** para el campo **CVV** con la propiedad **decoration** el **CustomInputDecoration** del control personalizado. Como se muestra en la figura siguiente.

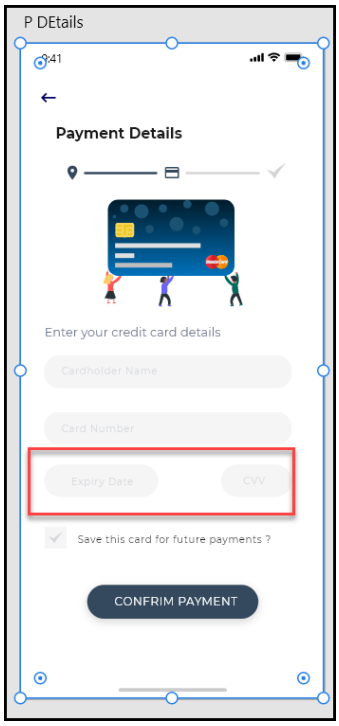
payment_details_screen.dart M

```
128                 const SizedBox(height: 10),
129                 Row(
130                   children: [
131                     Expanded(
132                         child: TextFormField(
133                             decoration:
134                                 CustomInputDecoration('Fecha de expiración'))), // TextFormField
135                     const Expanded(child: SizedBox(width: 20)),
136
137                     Expanded(
138                       child: TextFormField(
139                         decoration:
140                             CustomInputDecoration('CVV'),
141                       ), // TextFormField
142                     ), // Expanded       You, 31 seconds ago • Uncommitted changes
143
144                   ],
145                 ), // Row
146               ],
147             ), // ListView
148           )), // Padding // Form
```

14. **Ejecute** y **pruebe** le proyecto.



15. Si lo comparamos con el **mockup** podemos visualizar que el primer **TextFormField** (**Fecha de expiración**) tiene más o menos el **doble del espacio** que el segundo **TextFormField** (**cvv**),



16. Para tener el diseño que se muestra en el mockp utilice la propiedad **flex** de los **dos Expanded** tal y como se muestra en el siguiente código

```dart
payment_details_screen.dart M
                focusNode: cardNumberFocus,
                textInputAction: TextInputAction.next,
              ), // TextFormField
              const SizedBox(height: 10),
              Row(
                children: [      You, 1 second ago • Uncommitted changes
                  Expanded(

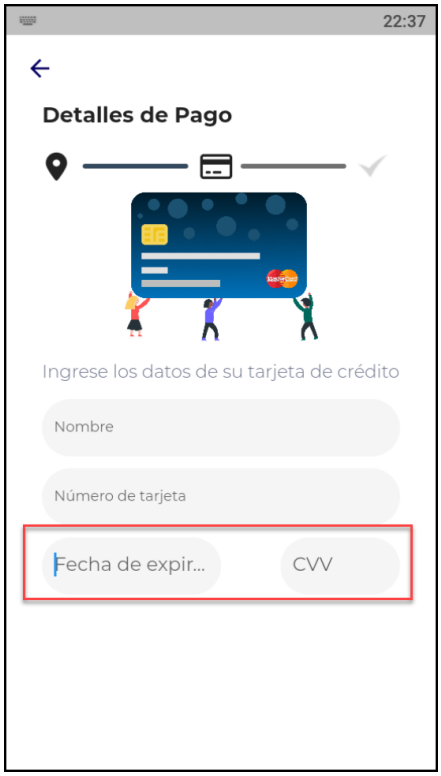
                    flex: 3,

                    child: TextFormField(
                      decoration: CustomInputDecoration('Fecha de expiración'),
                    ), // TextFormField
                  ), // Expanded
                  const Expanded(child: SizedBox(width: 20)),
                  Expanded(

                    flex: 2,

                    child: TextFormField(
                      decoration: CustomInputDecoration('cvv'),
                    ), // TextFormField
                  ), // Expanded
                ],
              ) // Row
            ],
          ), // ListView
```

17. **Ejecute** y **pruebe** el proyecto.



18. Agregue para el **TextFormField** (**Fecha de expiración**) un **estilo** (propiedad **style**) con un tamaño de **fuente** específico, una **máscara** (propiedad **inputFormatters**) para que se ingrese los datos en formato de **fecha** y solo **números**, y que se active **solo las teclas** para ingresar fechas en el móvil (propiedad **keyboardType**).

payment_details_screen.dart M

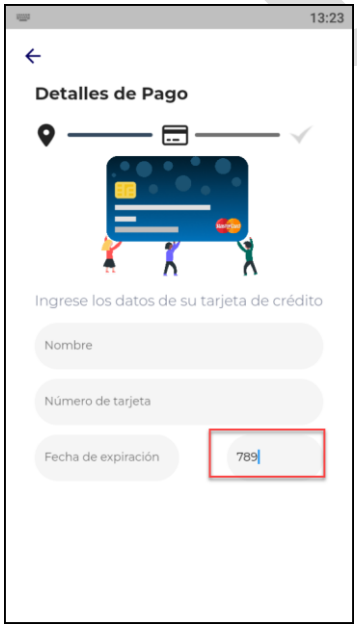
```
145                Row(
146                  children: [
147                    Expanded(
148                      flex: 3,
149
150                      child: TextFormField(
151
152                        style: const TextStyle(color: Colors.black, fontSize: 12),
153                        inputFormatters: <TextInputFormatter>[
154                          MaskTextInputFormatter(mask: '##/##')
155                        ], // <TextInputFormatter>[]
156                        keyboardType: TextInputType.datetime,
157
158                        decoration: CustomInputDecoration('Fecha de expiración'),
159
160                      ), // TextFormField       You, 1 second ago • Uncommitted changes
161                    ), // Expanded
162                    const Expanded(child: SizedBox(width: 20)),
163                    Expanded(
164                      flex: 2,
165                      child: TextFormField(
```

19. Para el **TextFormField** (**cvv**) agregue un **estilo** (propiedad **style**) con un tamaño de **fuente** específico, una **máscara** (propiedad **inputFormatters**) para que se ingrese solo números (**FilteringTextInputFormatter.digitsOnly**) y máximo 3 dígitos (**LengthLimitingTextInputFormatter(3)**), y que se active únicamente las teclas numéricas del móvil (propiedad **keyboardType**).

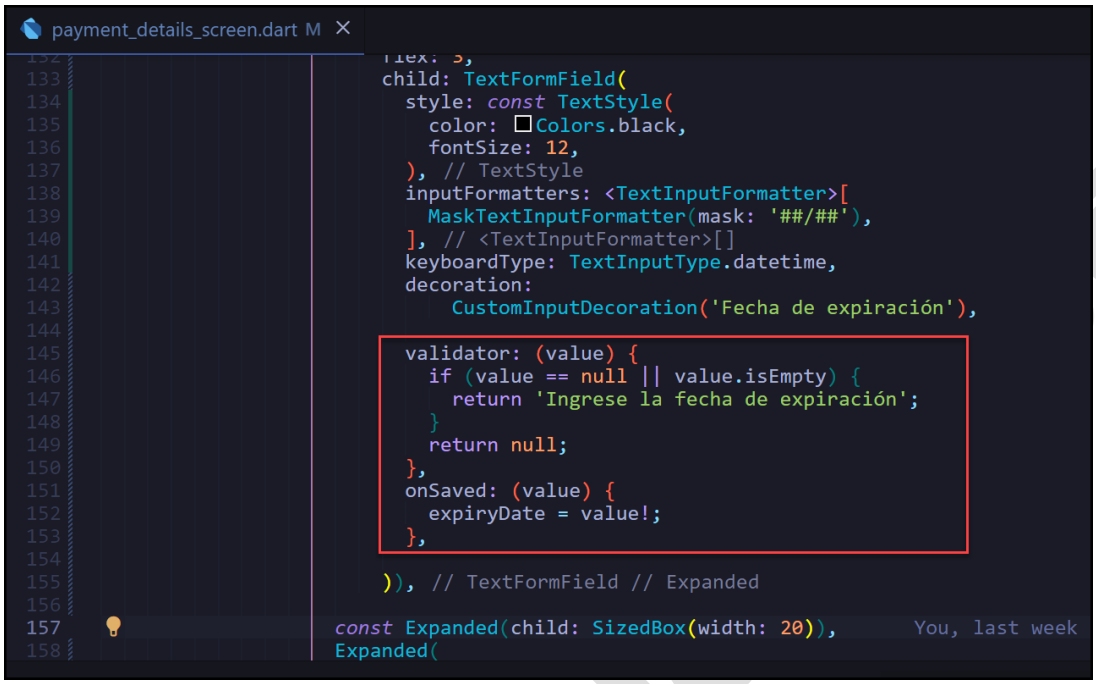
payment_details_screen.dart M

```
150                       inputFormatters: <TextInputFormatter>[
151                         MaskTextInputFormatter(mask: '##')
152                       ], // <TextInputFormatter>[]
153                       keyboardType: TextInputType.datetime,
154                       decoration: CustomInputDecoration('Fecha de expiración'),
155                     ), // TextFormField
156                   ), // Expanded
157                   const Expanded(child: SizedBox(width: 20)),
158                   Expanded(
159                     flex: 2,
160                     child: TextField(       You, 6 seconds ago • Uncommitted changes
161
162                       style: const TextStyle(color: Colors.black, fontSize: 12),
163                       inputFormatters: <TextInputFormatter>[
164                         FilteringTextInputFormatter.digitsOnly,
165                         LengthLimitingTextInputFormatter(3)
166                       ], // <TextInputFormatter>[]
167                       keyboardType: TextInputType.number,
168
169                       decoration: CustomInputDecoration('cvv'),
170                     ), // TextField
171                   ), // Expanded
172                 ],
173               ) // Row
174             ],
175           ), // ListView
```

20. **Ejecute** y **pruebe** el proyecto.



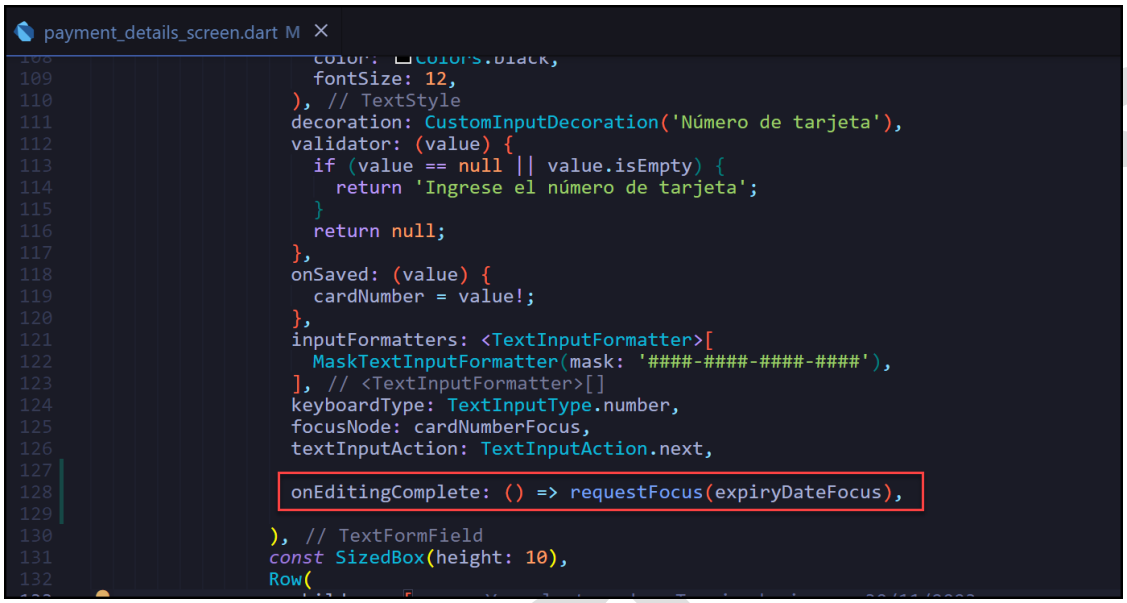
21. A continuación, para el **TextFormField** (**Fecha de expiración**) agregue las propiedades **validator** para **validar** los **datos ingresados** y la propiedad **onSaved** para guardar el estado (**información**) de los **datos ingresados** en el **TextFormField**.



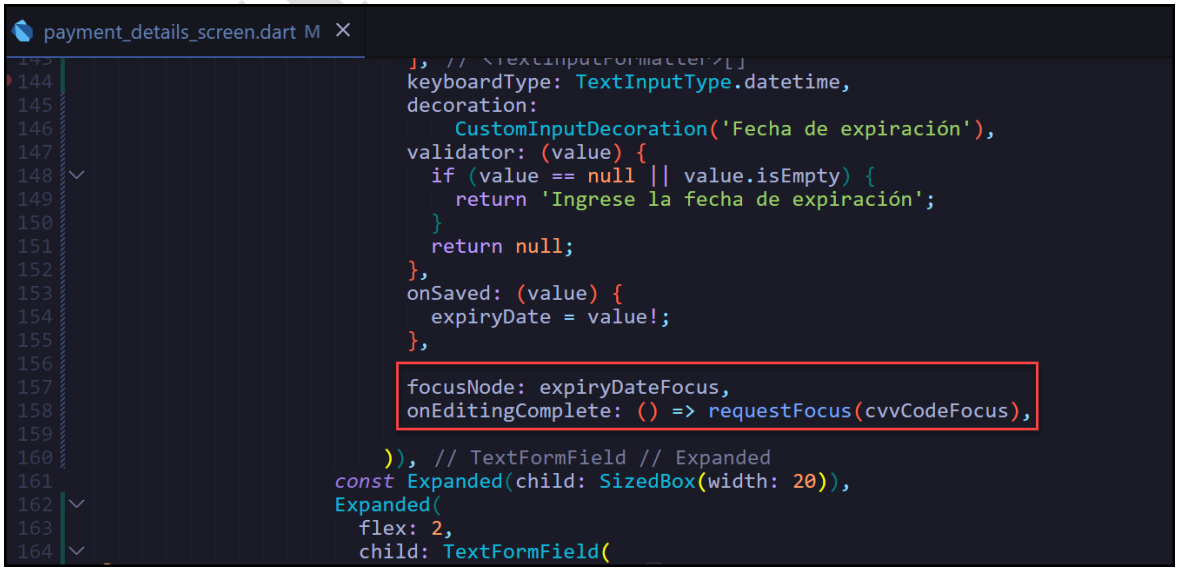
22. Repetimos el proceso para el **TextFormField** (**cvv**), por lo tanto, agregue las propiedades **validator** para **validar** los **datos ingresados** y la propiedad **onSaved** para guardar el estado (**información**) de los **datos ingresados** en el **TextFormField**.



23. Ahora vamos a trabajar con los **Focus** de cada **TextFormField**. Esto ya lo hicimos anteriormente con el **TextFormField** del "**Nombre**", es decir usamos la propiedad **onEditingComplete**, la cual se ejecuta cuando se presiona el **action** del teclado del móvil. Por lo tanto, **copie** y **pegue** el **código** del **TextFormField** del '**Nombre**' para el **TextFormField** del "**Número de tarjeta**", el cual quedaría de la siguiente forma para poner el **foco** en el **siguiente TextFormField**.



24. Ahora **repetimos** el proceso para poner el **foco** en el **TextFormField** de la "**Fecha de expiración**", y luego que se termine de editar poner el **foco** en el **siguiente TextFormField** (**cvv**).



25. Para el **TextFormField** del "**cvv**" **únicamente** ponemos el foco puesto que es el último **TextFormField** del formulario.

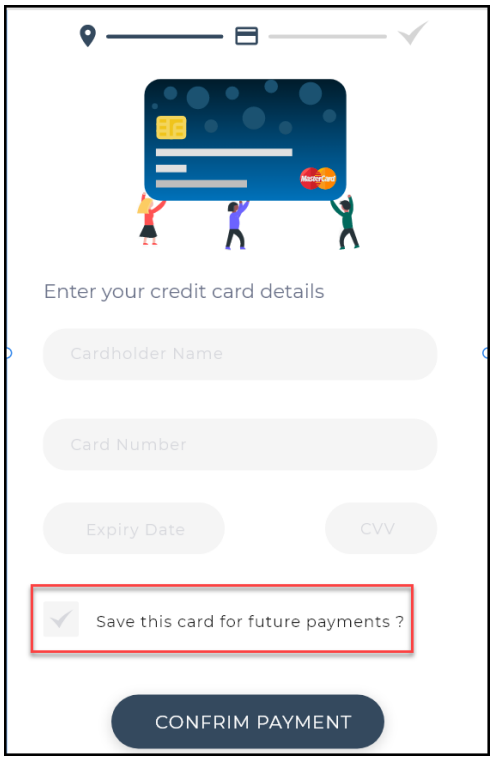
payment_details_screen.dart M

```
158                 Expanded(
159                   flex: 2,
160                   child: TextFormField(
161                     style: const TextStyle(
162                       color: Colors.black,
163                       fontSize: 12,
164                     ), // TextStyle
165                     inputFormatters: <TextInputFormatter>[
166                       FilteringTextInputFormatter.digitsOnly,
167                       LengthLimitingTextInputFormatter(3)
168                     ], // <TextInputFormatter>[]
169                     keyboardType: TextInputType.number,
170                     decoration: CustomInputDecoration('CVV'),
171                     validator: (value) {
172                       if (value == null || value.isEmpty) {
173                         return 'Ingrese el código cvv';
174                       }
175                       return null;
176                     },
177                     onSaved: (value) {
178                       cvvCode = value!;
179                     },
180
181                     focusNode: cvvCodeFocus,
182
183                   ), // TextFormField      You, last week • Terminado jueves 30/11/2023
184                 ), // Expanded
185               ],
186             ), // Row
```

26. **Ejecute** y **prueba** el proyecto.



27. Vamos a continuar con la siguiente sección del **mockup**, es decir el **checkbox** (“**Save this card for future payments** ?”), esto se muestra en la siguiente figura.



28. Por lo tanto, primero **agregue** un espacio en blanco con un **SizeBox** luego del **Row**.

```
payment_details_screen.dart M
                    return 'Ingrese el código cvv';
                  }
                  return null;
                },
                onSaved: (value) {
                  cvvCode = value!;
                },
                focusNode: cvvCodeFocus,
              ), // TextFormField
            ), // Expanded
          ],
        ), // Row

        const SizedBox(height: 10,),

      ],
    ), // ListView
  )), // Padding // Form      You, 2 months ago • Terminado domingo 22 de octubre
  ); // Scaffold
}

@override
void initState() {
  super.initState();
```

29. Según el **mockup**, ahora **agregue** otro **Row** para poner dentro un **checkBox** y un **Text**, Dentro del **checkBox** establecemos el valor (**value = savedCard**), esto para cuando se presione en el **checkBox** (propiedad **onChanged**) se cambie el valor de **savedCard**.



30. Como se puede apreciar en la figura **anterior**, el código genera un **error**, ya que la variable **savedCard** no puede modificarse puesto que es de tipo **final**, por lo que hay que quitar la palabra reservada **final** de la variable **savedCard**, como se muestra a continuación.

Antes:

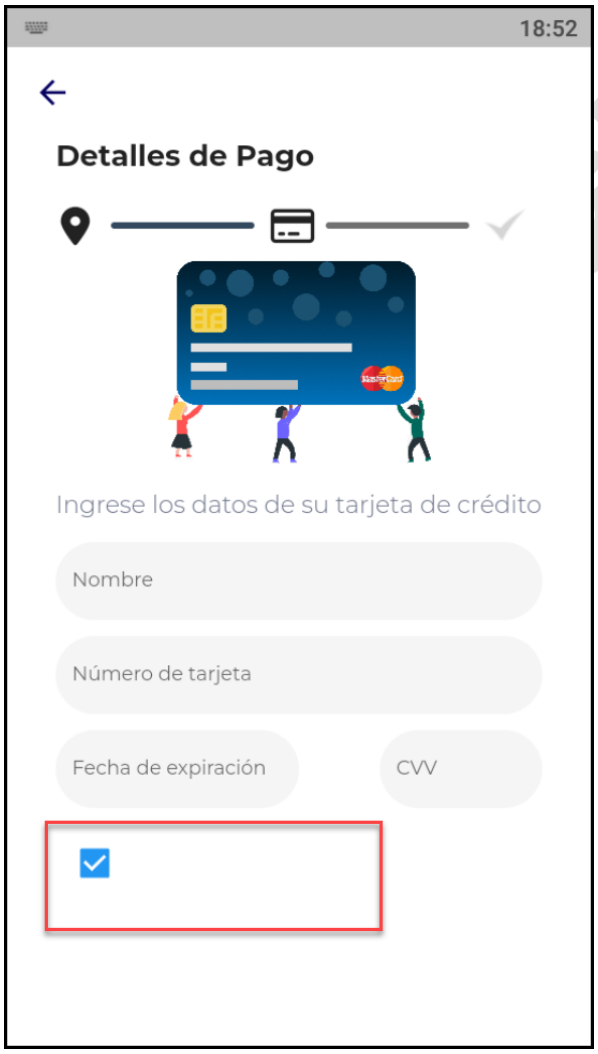


Después de cambiar la declaración de la variable al inicio de la clase **_PaymentDetailsScreenState**:

```
payment_details_screen.dart M
18   final _formKey = GlobalKey<FormState>();
19
20   String name = '';
21   String cardNumb
22   String expiryDa
23   String cvvCode
24
25   late FocusNode
26   late FocusNode expiryDateFocus;
27   late FocusNode cvvCodeFocus;
28
29   bool savedCard = false;
30
31   @override
32   Widget build(BuildContext context) {

FocusNode expiryDateFocus
Type: FocusNode
package:payment_flow/ui/screens/payment_details_screen.dart
royecto 29 de octubre
```

31. **Ejecute** y **pruebe** el proyecto.



32. Luego del **CheckBox** dentro del **Row** añada un **Text** esto dentro de un control **Expanded** para que este control tome todo el espacio disponible en la fila.

```
payment_details_screen.dart M
                            cvvCode = value!;
                          },
                          focusNode: cvvCodeFocus,
                        ), // TextFormField
                      ), // Expanded
                    ],
                  ), // Row
                  const SizedBox(
                    height: 10,
                  ), // SizedBox
                  Row(
                    children: [      You, 1 minute ago • Uncommitted changes
                      Checkbox(
                          value: savedCard,
                          onChanged: ((value) {
                            setState(() {
                              savedCard = value!;
                            });
                          })), // Checkbox

                      const Expanded(
                        child: Text('Guardar esta tarjeta para futuras compras?'),
                      ), // Expanded

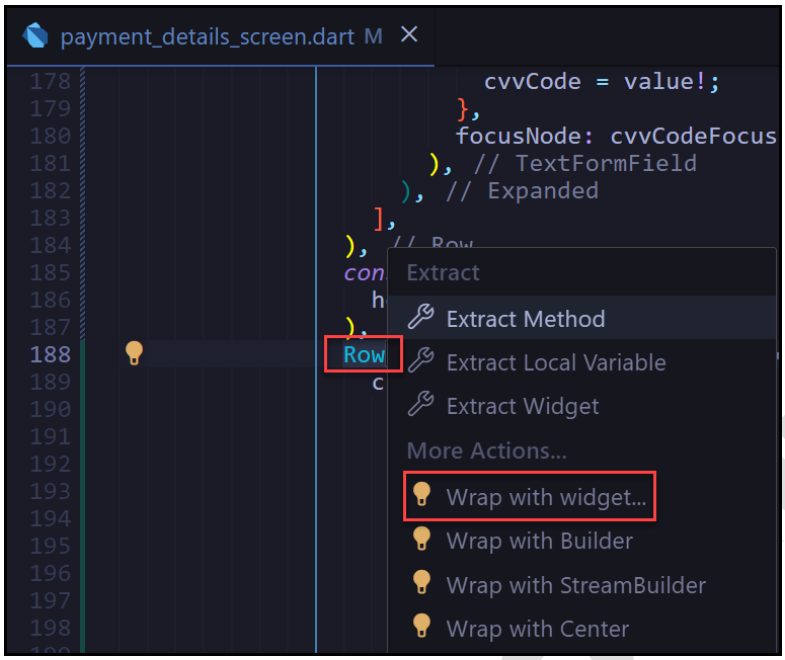
                    ],
                  ) // Row
                ],
              ), // ListView
            )), // Padding // Form
```

33. **Ejecute** y **pruebe** el proyecto.

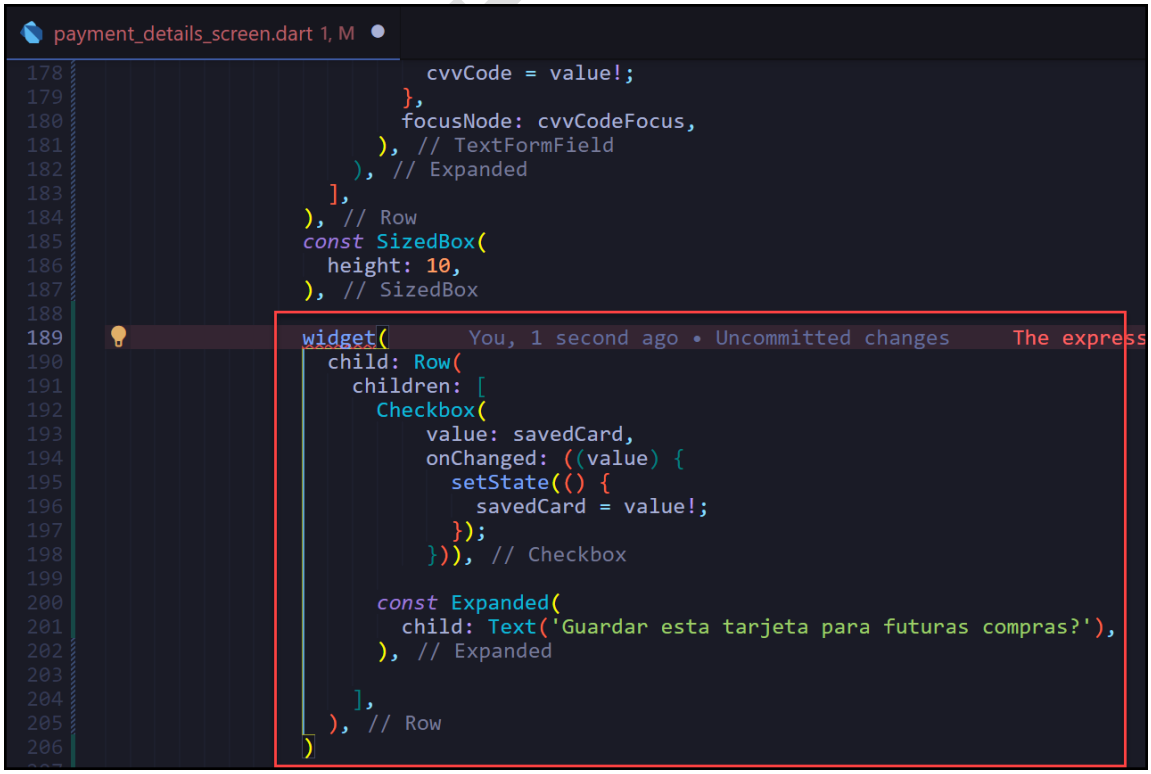


34. Para darle un **efecto especial** al seleccionar el **checkBox** y al **texto**, vamos a **envolver** el **Row** en un widget **InkWell**. Para esto **ponga el cursor** dentro de la palabra **Row** (el último Row) y **presione las teclas Control + Punto** (Control + .), de la lista emergente **seleccione** la opción **Wrap with widget**.

Ponga el mouse en la palabra **Row** y de la lista emergente **seleccione** la opción indicada.



El **Row** se envuelve con un **widget**.

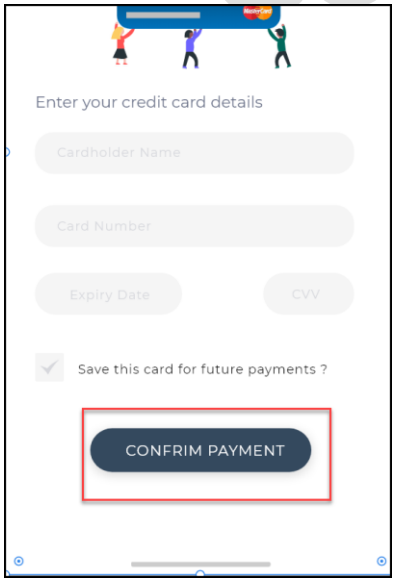


35. **Cambie** la palabra **widget** generada anteriormente por la palabra **InkWell**, agregue el evento **onTap** del **InkWell** para **cambiar el estado** (**savedCard**), es decir si está en **true** cambia a **false** y si está en **false** cambia a **true**. Al utilizar el **InkWell** permite que se capture el **evento para toda la fila** y otro evento que **captura el evento del checkBox**.

payment_details_screen.dart M

```
189                     focusNode: ccvCodeFocus,
190                   ), // TextFormField
191                 ), // Expanded
192               ],
193             ), // Row
194             const SizedBox(
195               height: 10,
196             ), // SizedBox
197             InkWell(      You, 21 seconds ago • Uncommitted changes
198
199               onTap: () {
200                 setState(() {
201                   savedCard = !savedCard;
202                 });
203               },
204
205               child: Row(
206                 children: [
207                   Checkbox(
208                       value: savedCard,
209                       onChanged: ((value) {
```

36. Ahora de acuerdo con el **mockup** hay que **agregar** el **botón** para **guardar el pago** (**CONFIRM PAYMENT**).



37. **Primero** hay agregue un **espacio en blanco** con un **SizeBox** luego del **InkWell** anterior que contiene el **Row**.

```
payment_details_screen.dart M
199                         setState(() {
200                           savedCard = value!;
201                         });
202                       })), // Checkbox
203                     const Expanded(
204                       child:
205                           Text('Guardar esta tarjeta para futuras compras?'),
206                     ), // Expanded
207                   ],
208                 ), // Row
209               ), // InkWell       You, 2 minutes ago • Uncommitted changes
210
211               const SizedBox(height: 10,),
212
213             ],
214           ), // ListView
215         )), // Padding // Form
216       ); // Scaffold
217     }
218
219     @override
220     void initState() {
221       super.initState();
```

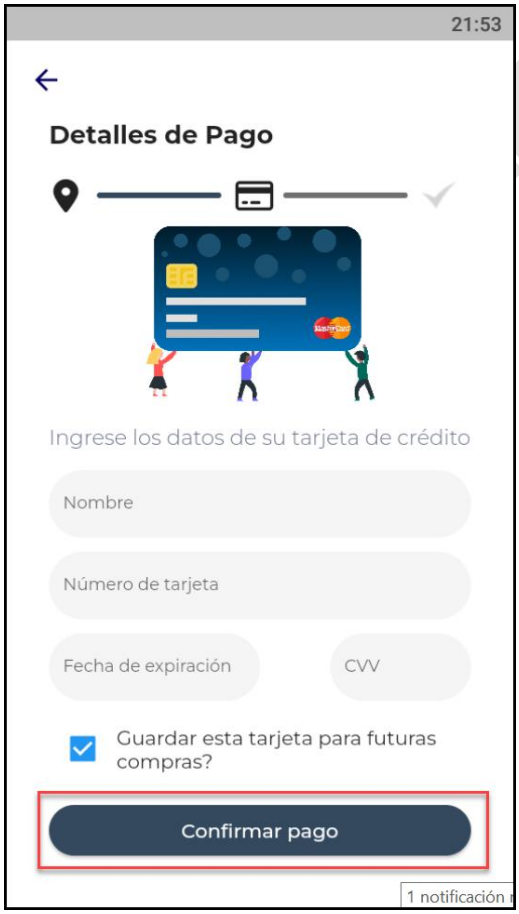
38. **Luego** del **SizeBox** agregue un **ElevatedButton**, con el texto "**Confirmar pago**", un **borde circular** y el evento **onPressed** que ejecute un método llamado **showSuccessful** el cual se va a crear en el próximo punto.

```
payment_details_screen.dart 1, M
210                         });
211                       })), // Checkbox
212                     const Expanded(
213                       child: Text('Guardar esta tarjeta para futuras compras?'),
214                     ), // Expanded
215                   ],
216                 ), // Row
217               ), // InkWell
218               const SizedBox(height: 10),
219
220               ElevatedButton(       You, 5 seconds ago • Uncommitted changes
221                 onPressed: () {
222                   return showSuccessful(context);    The method 'showSuccessful' isn't defined
223                 },
224                 style: ElevatedButton.styleFrom(
225                     shape: RoundedRectangleBorder(
226                   borderRadius: BorderRadius.circular(25),
227                 )), // RoundedRectangleBorder
228                 child: const Text('CONFIRMAR PAGO'),
229               ), // ElevatedButton
230
231             ],
232           ), // ListView
233         ), // Padding
```

39. **Agregue** al final de la clase **_PaymentDetailsScreenState** el método **showSuccessful** por ahora vació.

```
payment_details_screen.dart M
248     }
249
250     requestFocus(FocusNode focusNode) {
251       FocusScope.of(context).requestFocus(focusNode);
252     }
253
254     void showSuccessful(BuildContext context) {
255
256     }
257
258   }
259
      You, 2 months ago | 1 author (You)
260   class CustomInputDecoration extends InputDecoration {
261     final String inputText;        You, 2 months ago • Terminado domingo 22 de octubre
262
```

40. **Ejecute** y **pruebe** el proyecto.



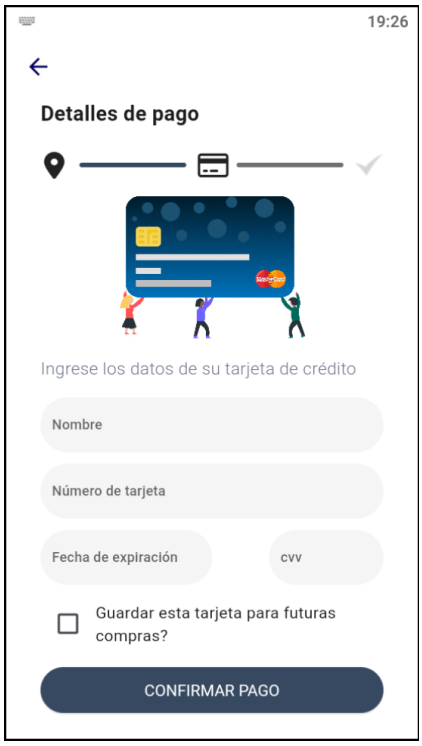
Nota	• Note que el **ElevatedButton** no se ha ubicado en la propiedad **floatingActionButton** como si se lo hizo en el primer **formulario,** puesto que no se desea que se muestre por defecto siempre en esa posición, es decir al **final del formulario,** para este segundo formulario no importa si se hace o no un **scroll** para poder visualizar el botón (CONFIRMAR PAGO).

41. Vamos a **programar** el código del método **showSuccessful**, lo primero que vamos a realizar es **comprobar** que el formulario pase todas las **validaciones** establecidas anteriormente, si es así entonces **guardamos el estado del formulario** (la información) y luego **mostraremos** la siguiente **ventana** (esto lo haremos más adelante), por ahora cree el siguiente código para el método.

```
payment_details_screen.dart M
248     }
249
250     requestFocus(FocusNode focusNode) {
251       FocusScope.of(context).requestFocus(focusNode);
252     }
253
254     void showSuccessful(BuildContext context) {
255       if (formKey.currentState!.validate()) {
256         formKey.currentState?.save();
257       }
258     }
259
260   }
261         You, 2 months ago • Terminado domingo 22 de octubre
      You, 2 months ago | 1 author (You)
262   class CustomInputDecoration extends InputDecoration {
263     final String inputText;
```

42. **Ejecute** y **compruebe** que el formulario pasa todas las **validaciones** establecidas en el código anteriormente.

Se **visualiza** el formulario.

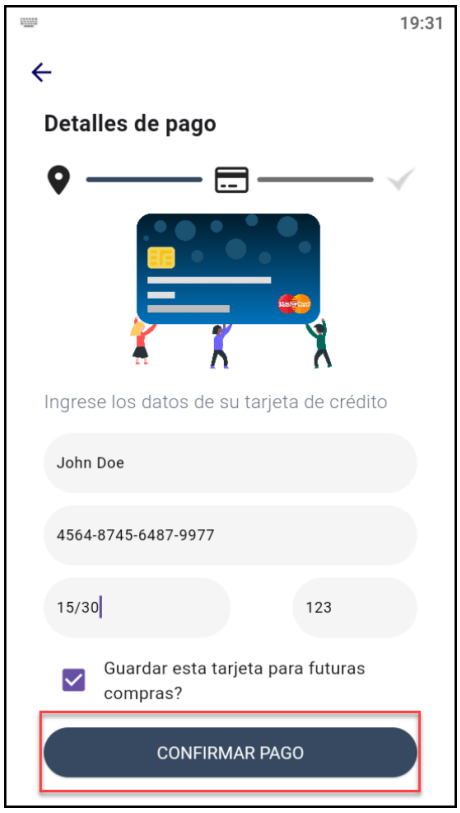


Presione el botón para que se ejecute las **validaciones**.



Clay Aldás

Ingrese todos los datos en los controles y presione el botón.



43. Ahora **abra** el archivo **main.dart** y al final del archivo, en la propiedad **routes** agregue una nueva ruta, por ahora se va a mostrar en la ruta el segundo formulario **PaymentDetailsScreen** hasta crear el formulario **correcto** (tercer formulario) más adelante, el código se muestra continuación.

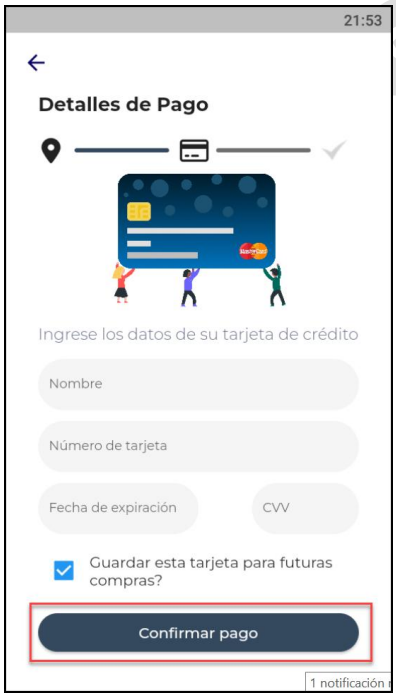


44. Vamos a agregar código al método **showSuccessful** anterior, puesto que una vez que el usuario **confirma el pago** el botón para poder ir hacia atrás **debe desaparecer** y se debe **borrar todas las ventanas de navegación** del historial. Esto evita que el usuario por ejemplo **inicie sesión** si ya se inició sesión, o si acaba de **cerrar sesión** se debe borrar el **historial de navegación**, y así evitar que el usuario **vaya hacia atrás** con los botones correspondientes para esto y cometa errores en este tipo de pantallas. Para esto en el archivo **payment_details_screen.dart**, en la clase **_PaymentDetailsScreenState**, en el método **showSuccessful** agregue el siguiente código.

payment_details_screen.dart M

```
255     FocusScope.of(context).requestFocus(focusNode);
256   }
257
258   void showSuccessful(BuildContext context) {
259     if (formKey.currentState!.validate()) {
260       formKey.currentState?.save();
261
262       Navigator.pushNamedAndRemoveUntil(context, '/payment_successful',
263           ModalRoute.withName('/payment_successful'));
264     }
265   }
266 }       You, yesterday • Terminada practica 009
```

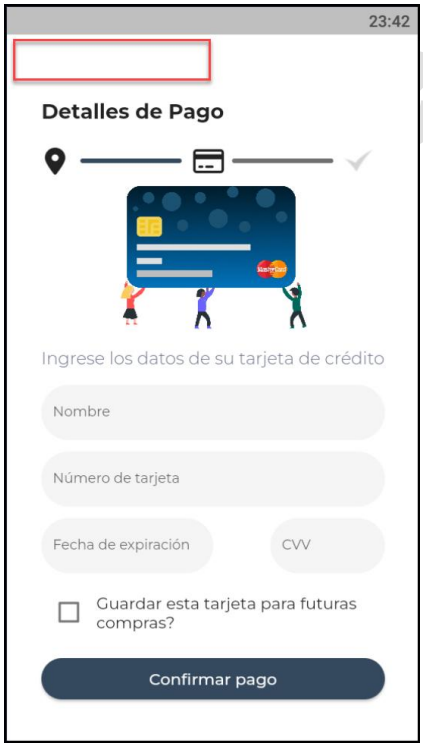
45. **Ejecute** y **pruebe** el proyecto.



46. **Ingrese** todos los datos en el formulario para que se ejecute la validación.



47. **Presione** en el botón Confirmar pago.



Clay Aldás

48. Fíjese que una vez que se pasa todas las **validaciones** y se presione el botón **Confirmar pago**, el **botón** de ir **hacia atrás** y todos los botones de **navegación no están activados en el móvil**, esto es muy importante en este tipo de formularios. Por ahora se muestra el formulario **PaymentDetailsScreen**, pero en la próxima práctica se va a mostrar el formulario adecuado (tercer formulario).

Lecturas útiles para revisar y profundizar en los temas abordados en esta práctica.

FocusScope class

https://api.flutter.dev/flutter/widgets/FocusScope-class.html

Navigator class

https://api.flutter.dev/flutter/widgets/Navigator-class.html

ModalRoute class

https://api.flutter.dev/flutter/widgets/ModalRoute-class.html

flutterClay Aldás